

Ambiance des lieux — Phase 2

Consommation et visualisation de l'API de collecte

IFT3225 — Technologies internet
Université de Montréal

Équipe

Aguibou FOFANA — 20332292
Mamadou TRAORE — 20290120
Kofi OSEL — 20272184

19 juillet 2026

1. Conception de l'interface

L'application cliente est une application React (Vite) monopage qui consomme l'API REST de la Phase 1, mise à jour pour la Phase 2. Elle s'articule autour des deux vues complémentaires exigées par l'énoncé : une vue carte qui donne une lecture d'ensemble de l'ambiance des lieux instrumentés, et une vue détaillée qui dresse le portrait complet d'un lieu (classification courante, historique, créneaux calmes, contribution d'observations).

1.1 Décomposition en composants

L'interface est découpée en composants à responsabilité unique, conformément à l'exigence d'organisation modulaire :

Composant	Responsabilité
App.jsx	État global : lieu sélectionné, session utilisateur (token JWT), favoris, navigation entre vues, déconnexion automatique en cas de session expirée.
MapView.jsx	Carte Leaflet/OpenStreetMap : marqueurs positionnés par coordonnées, colorés selon la classification, popup résumé, clic vers la vue détaillée, légende, rafraîchissement temps réel des marqueurs (SSE).
LocationDetail.jsx	Portrait d'un lieu : badge de classification, graphe d'historique (Chart.js), créneaux calmes, formulaire d'observation (si connecté), bouton favori, mise à jour en direct (SSE).
LoginForm.jsx / RegisterForm.jsx	Formulaires de connexion et de création de compte, avec gestion des erreurs renvoyées par l'API.
MyLocations.jsx	Vue « Mes lieux » : récapitulatif des lieux où l'utilisateur connecté a soumis des observations (nombre d'écoutes, dernière contribution, accès au détail, gestion du favori).
api/ambianceApi.js	Couche client : seule zone du code qui parle à l'API (axios) et au flux temps réel (EventSource). Aucun composant n'appelle l'API directement.

1.2 Parcours utilisateur

À l'ouverture, l'utilisateur voit la carte des lieux et leur ambiance courante sans aucune authentification (accès public). Un clic sur un marqueur ouvre le portrait détaillé du lieu. L'en-tête reflète en permanence l'état de session : bouton « Connexion » pour un visiteur ; salutation, filtre « Mes favoris », accès à la vue « Mes lieux » et bouton « Déconnexion » pour un utilisateur connecté. Les actions protégées (soumettre une observation, gérer les favoris) ne sont affichées que si l'utilisateur est connecté, conformément à l'exigence d'affichage/masquage selon les permissions.

1.3 Gestion explicite des états

Chaque vue distingue explicitement trois états, comme exigé : chargement (indicateur « Chargement... » pendant les appels), erreur (message dédié si l'API est injoignable ou renvoie une erreur) et vide (« Aucune mesure disponible sur les dernières 24 heures » pour l'historique, « Aucun créneau calme détecté » pour les créneaux, marqueur gris « Données non disponibles » sur la carte). Une bannière d'information signale en outre les échecs d'actions (favoris) et l'expiration de session.

2. Consommation de l'API

2.1 Couche client isolée

Tous les appels HTTP passent par `client/src/api/ambianceApi.js`. Cette couche centralise l'URL de base (configurable via la variable d'environnement `VITE_API_URL`, fichier `client/.env` — un `.env.example` est fourni), la construction des requêtes, l'injection de l'en-tête `Authorization: Bearer <token>` pour les actions protégées, l'abonnement au flux temps réel (SSE) et un intercepteur qui détecte les tokens expirés (401) pour déconnecter proprement l'utilisateur. Les composants ne connaissent ni axios ni les URLs : changer d'API ou de transport ne toucherait que ce fichier.

2.2 Endpoints consommés

Endpoint	Usage dans le client	Auth
GET <code>/v1/locations</code>	Liste des lieux et coordonnées pour la carte	publique
GET <code>/v1/ambiance/{slug}/now?window=30m</code>	Badge de classification (carte et vue détaillée)	publique
GET <code>/v1/ambiance/{slug}/history?last=24h&bucket=1h</code>	Graphe d'historique du niveau sonore	publique
GET <code>/v1/ambiance/{slug}/quiet-hours?days=7</code>	Créneaux calmes de la semaine	publique
GET <code>/v1/events</code> (SSE)	Flux temps réel : rafraîchit carte et portrait sans rechargement (bonus)	publique
POST <code>/v1/auth/register</code>	Création de compte	publique
POST <code>/v1/auth/login</code>	Connexion (obtention du JWT)	publique
GET/POST/DELETE <code>/v1/auth/favorites</code>	Consultation et gestion des lieux favoris	JWT
GET <code>/v1/auth/my-locations</code>	Vue « Mes lieux » : lieux où l'utilisateur a soumis des observations	JWT
POST <code>/v1/observations/user</code>	Soumission d'une observation liée à l'auteur	JWT

2.3 Format des réponses et gestion des erreurs

L'API renvoie une enveloppe uniforme : `{ status, data, meta }` en succès et `{ status: "error", error: { code, message, details? }, meta }` en erreur. Les messages d'erreur sont descriptifs et en français, ce qui permet au client de les afficher tels quels sans documentation externe (exigence de réponses auto-descriptives). Côté client, chaque appel est encadré : les erreurs réseau ou HTTP alimentent l'état erreur de la vue, et les erreurs de validation des formulaires affichent `error.message` renvoyé par le serveur (ex. champ manquant, identifiants invalides, valeur hors domaine). Les codes utilisés : 400 (validation), 401 (token absent/expiré, identifiants invalides), 403 (interdit), 404 (lieu inconnu), 409 (conflit à l'inscription), 422 (valeur hors domaine).

3. Visualisation

3.1 Classification et justification des seuils

La classification est calculée exclusivement côté serveur (fonction `ambianceLabel`) à partir du niveau sonore moyen sur la fenêtre d'analyse ; le client ne fait qu'afficher le badge reçu. Les seuils retenus s'appuient sur des repères acoustiques usuels :

Classification	Seuil (dB)	Justification
calme	< 50	Niveau d'une bibliothèque ou d'une conversation feutrée : propice au travail.
modéré	50 – 65	Conversation normale, bureau vivant : bruit de fond présent mais non gênant.
animé	65 – 75	Cafétéria à l'heure de pointe, rue passante : ambiance sociale marquée.
bruyant	> 75	Niveau où la conversation devient difficile et l'exposition prolongée fatigante.
inconnu	—	Aucune mesure dans la fenêtre : affiché en gris « Données non disponibles ».

3.2 Unités et échelles

L'unité unique est le décibel (dB), seule grandeur mesurée par le capteur Phyphox (type `noise_level`, plage validée 0–140 dB). Le graphe d'historique affiche la moyenne par tranche horaire sur 24 heures, avec axes titrés (« Niveau sonore (dB) » en ordonnée, « Heure » en abscisse). L'échelle verticale est **adaptive** : les bornes sont calées sur les valeurs effectivement mesurées, avec une marge de ± 5 dB, tout en restant confinées à la plage validée par l'API (0–140 dB). Un axe fixe 0–140 écraserait la plage réellement utile (typiquement 40–80 dB) et rendrait les variations illisibles ; les bornes s'ajustent donc à chaque lieu pour maximiser la lisibilité, et l'axe titré garde la lecture sans ambiguïté d'un lieu à l'autre. Les tranches horaires sans mesure apparaissent comme des trous dans la courbe plutôt que comme 0 dB, afin de ne pas confondre absence de données et silence total. Les créneaux calmes présentent, pour chaque tranche de 30 minutes sous le seuil de 55 dB sur les 7 derniers jours, le jour, la plage horaire et le niveau moyen mesuré — l'échelle est donc toujours visible à côté de la valeur.

3.3 Seuil de fraîcheur (vue carte)

Le badge de la carte est calculé sur une fenêtre glissante de 30 minutes (`/now?window=30m`). Ce seuil de fraîcheur a été choisi car l'ambiance d'un lieu évolue à l'échelle de la demi-heure (pause de midi, fin de cours) : une fenêtre plus courte rendrait la classification instable avec peu de mesures, une fenêtre plus longue masquerait les transitions. Au-delà de 30 minutes sans mesure, le serveur renvoie « inconnu » et le marqueur passe en gris « Données non disponibles » : la carte reste lisible sans mesures récentes et n'affiche jamais une ambiance périmée comme si elle était actuelle. Le flux temps réel (bonus SSE, section 7.2) complète ce mécanisme : dès qu'une nouvelle mesure arrive, le badge du lieu concerné est recalculé et réaffiché sans rechargement.

3.4 Codage couleur

Un même code couleur est utilisé sur la carte, dans la légende et sur les badges : vert (calme), orange (modéré), rouge (animé), violet (bruyant), gris (données non disponibles). La légende est affichée en permanence sous la carte.

4. Authentification et espace compte

4.1 Mécanisme

L'authentification utilisateur repose sur des tokens JWT signés côté serveur (JWT_SECRET, expiration 7 jours). Les mots de passe sont hachés avec bcrypt avant stockage (jamais en clair), avec une longueur minimale de 6 caractères validée à l'inscription. La connexion accepte le nom d'utilisateur ou l'adresse courriel. Le token et le profil sont conservés dans le localStorage pour maintenir la session entre les visites ; la déconnexion les efface. Si le serveur rejette le token (expiré ou invalide), un intercepteur de la couche client déconnecte automatiquement l'utilisateur et affiche « Votre session a expiré. Veuillez vous reconnecter. » — aucune action ne peut donc échouer silencieusement.

4.2 Actions protégées

Le middleware serveur userAuth vérifie l'en-tête Authorization: Bearer <token> et renvoie 401 (NO_TOKEN / INVALID_TOKEN) si le token est absent, invalide ou expiré. Sont protégés : la soumission d'observation (POST /v1/observations/user, qui enregistre automatiquement l'utilisateur comme author et horodate côté serveur) et la gestion des favoris. Côté client, ces actions sont masquées pour les visiteurs non connectés ; l'authentification device de la Phase 1 (x-api-key) reste inchangée pour la collecte capteur.

4.3 Espace compte

Une fois connecté, l'utilisateur dispose de : la création/suppression de favoris depuis la vue détaillée, un filtre « Mes favoris » sur la carte pour retrouver ses lieux personnels, le formulaire d'observation (densité, proximité, ambiance ressentie, notes) avec confirmation de soumission et affichage des erreurs de validation du serveur, et la vue « Mes lieux » : un récapitulatif des lieux où il a effectué des écoutes, construit côté serveur (GET /v1/auth/my-locations, agrégation MongoDB des observations par lieu) qui affiche pour chaque lieu le nombre d'observations soumises, la date de la dernière contribution, le statut favori (modifiable sur place) et un accès direct au portrait détaillé. La liaison observation–auteur ajoutée à l'infrastructure (section 5) est ainsi exploitée de bout en bout.

5. Mise à jour de l'infrastructure (API Phase 1)

La contrainte directrice était de faire évoluer sans rompre : aucun endpoint de la Phase 1 n'a été modifié ni supprimé, et tous les nouveaux champs sont optionnels. La collecte par bridge Phyphox et les lectures existantes fonctionnent à l'identique.

Évolution	Détail	Rétrocompatibilité
Coordonnées des lieux	Champs latitude/longitude (validés -90..90 / -180..180) sur Location, exposés par GET /v1/locations.	Champs optionnels : les lieux existants restent valides.
Auteur des observations	Champ author (référence User) sur Observation, renseigné par POST /v1/observations/user.	Optionnel : les observations device (x-api-key) restent sans auteur.
Modèle User (nouveau)	username et email uniques, mot de passe bcrypt, favoriteLocations.	Nouvelle collection : aucun impact sur l'existant.
Routes /v1/auth/*	register, login, gestion des favoris (JWT).	Nouvelles routes montées à côté des anciennes.
POST /v1/observations/user	Soumission d'observation authentifiée par JWT, horodatée serveur.	S'ajoute au POST /v1/observations (device) inchangé.

Évolution	Détail	Rétrocompatibilité
Classification exposée	ambianceLabel (calme/modéré/animé/bruyant/inconnu) dans /now et /compare ; échelles via history et quiet-hours.	Ajout de champs dans la réponse : les clients Phase 1 ignorent simplement les nouveaux champs.
Flux temps réel GET /v1/events (bonus)	Diffusion SSE des nouvelles mesures et observations, filtre optionnel par lieu.	Nouvelle route en lecture seule : aucun contrat existant modifié.
Portage TypeScript (bonus)	Backend entièrement réécrit en TypeScript (modèles, couche données, middlewares, routes typés), compilé vers dist/.	Contrats HTTP strictement identiques : mêmes endpoints, mêmes enveloppes.

Le versionnement /v1 hérité de la Phase 1 garantit qu'une future rupture de contrat passerait par un préfixe /v2 sans casser les clients existants.

6. Réactivité et utilisabilité

Les trois appels de la vue détaillée (ambiance courante, historique, créneaux calmes) sont lancés en parallèle avec `Promise.all`, ce qui réduit le temps d'affichage au plus lent des trois plutôt qu'à leur somme. Sur la carte, l'ambiance de chaque lieu est également résolue en parallèle, et un échec sur un lieu n'empêche pas l'affichage des autres (le marqueur passe simplement en gris).

Le flux temps réel (SSE) prolonge cette réactivité : plutôt que d'interroger périodiquement le serveur (polling), le client reçoit un événement dès qu'une mesure ou une observation est enregistrée, et seul le lieu concerné est rafraîchi — en arrière-plan, sans écran de chargement ni rechargement de page. La vue détaillée affiche « Mis à jour en direct à HH:MM:SS » pour rendre cette mise à jour transparente pour l'utilisateur.

Côté utilisabilité : retour visuel immédiat sur chaque action (boutons désactivés avec libellé « Connexion... » pendant les requêtes, message de succès après soumission d'une observation, étoile pleine/vide pour l'état favori), messages d'erreur en français issus directement de l'API, légende permanente sous la carte, bouton « Retour à la carte » toujours visible dans la vue détaillée. Les listes déroulantes du formulaire d'observation reprennent exactement les vocabulaires contrôlés de l'API (densité, proximité, vibe), ce qui rend les erreurs 422 impossibles depuis l'interface.

7. Limites, bonus réalisés et évolutions

7.1 Limites connues

- **Détail des contributions** : la vue « Mes lieux » agrège les observations de l'utilisateur par lieu, mais n'affiche pas encore la liste détaillée de chaque observation individuelle ; il manque aussi un filtre par auteur sur `GET /v1/observations`.
- **Stockage du token** : le JWT est conservé dans `localStorage`, vulnérable à un XSS ; un cookie `httpOnly` serait plus sûr. Il n'y a pas de rafraîchissement de token, mais l'expiration est désormais gérée proprement : au premier 401, l'application déconnecte l'utilisateur et l'invite à se reconnecter.
- **Charge réseau de la carte** : le chargement initial fait un appel `/now` par lieu (N+1) ; l'endpoint `/v1/ambiance/compare` pourrait servir de base à un appel unique groupé. Le SSE élimine en revanche tout polling ensuite.

- **Fraîcheur implicite** : le seuil de 30 minutes est appliqué par le serveur ; l'heure de la dernière mesure n'est pas encore affichée par lieu, même si la vue détaillée horodate désormais chaque mise à jour reçue en direct.
- **POST /v1/devices toujours ouvert** : la faille volontaire documentée en Phase 1 demeure ; la correction proposée (clé admin) n'a pas encore été appliquée.
- **Tests** : la collection Postman couvre l'API, mais le client React n'a pas de tests automatisés.

7.2 Bonus réalisés

Backend TypeScript. L'ensemble du serveur a été porté en TypeScript strict : interfaces des modèles Mongoose (IDevice, ILocation, IMeasurement, IObservation, IUser avec méthode comparePassword typée), vocabulaires contrôlés exprimés en types littéraux (Density, Vibe, Proximity), couche données et utilitaires typés (ApiError, enveloppes de réponse, portraits d'ambiance), handlers Express typés et augmentation de types pour req.device / req.user. La compilation (tsc) produit dist/ et npm start compile puis démarre ; les contrats HTTP sont restés strictement identiques (vérifié sur les endpoints de santé, lieux, ambiance et authentification). Gain concret : le compilateur a révélé un bug latent de la version JavaScript — l'appel à errors.unauthorized() dans la route de connexion référençait une fonction inexistante, si bien qu'un mot de passe erroné provoquait une erreur 500 au lieu d'un 401 INVALID_CREDENTIALS. Le typage a permis de détecter et corriger ce cas avant la remise.

Temps réel (SSE). Un endpoint GET /v1/events diffuse en Server-Sent Events chaque nouvelle mesure ou observation ({ kind, locationSlug, at }), avec filtre optionnel ?locationSlug= et keepalive périodique. Côté serveur, les routes d'écriture (mesures unitaires, lots, observations device et utilisateur) publient sur un bus d'événements interne relayé aux clients connectés. Côté client, la couche ambianceApi expose un abonnement EventSource : la carte rafraîchit le marqueur du lieu concerné et la vue détaillée recharge badge et historique en arrière-plan, sans rechargement de page. SSE a été préféré à WebSocket car le flux est unidirectionnel (serveur vers client), compatible HTTP/CORS sans dépendance supplémentaire, avec reconnexion automatique native d'EventSource.

7.3 Évolutions envisagées

- **Espace compte enrichi** : étendre la vue « Mes lieux » avec la liste paginée des observations individuelles et des statistiques personnelles.
- **Comparaison de lieux** : exploiter /v1/ambiance/compare dans l'interface pour recommander le lieu le plus calme à un instant donné.
- **Affichage de la fraîcheur** : exposer lastMeasurementAt par lieu et afficher « mis à jour il y a X min » sur les marqueurs.
- **Sécurisation de POST /v1/devices** : appliquer la clé d'administration à la création d'appareils, comme proposé en Phase 1.